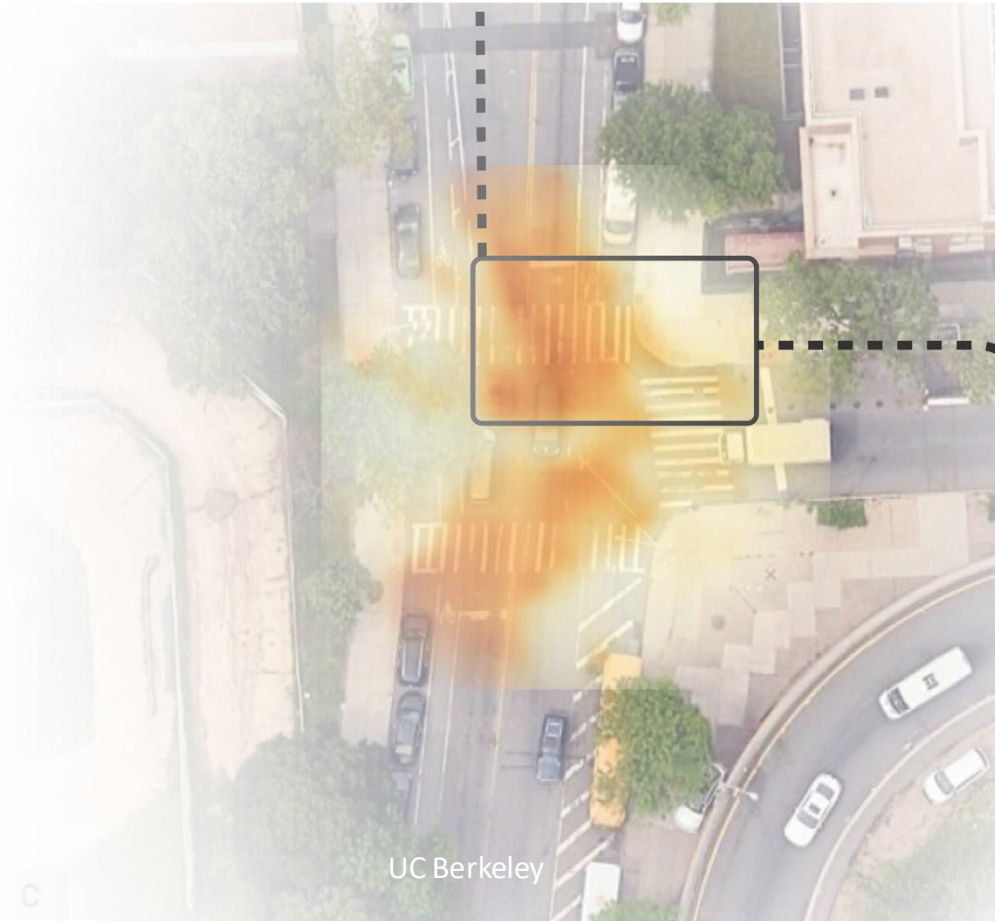
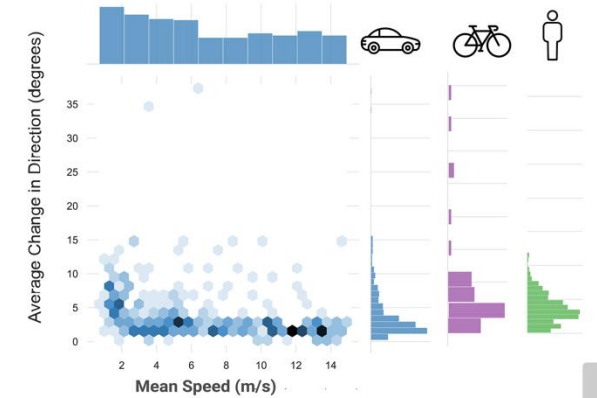
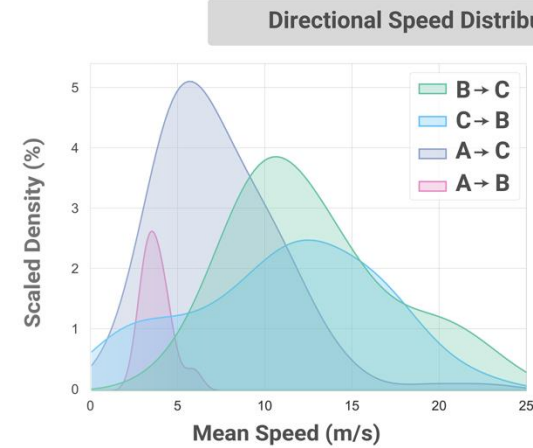
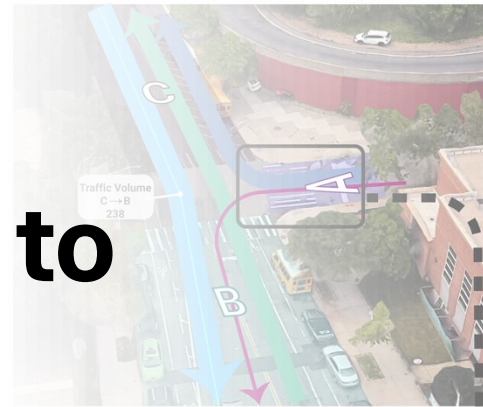


Introduction to Urban Data Analytics

Fall 2026

Maryam Hosseini

City & Regional Planning



UC Berkeley

C

B

D

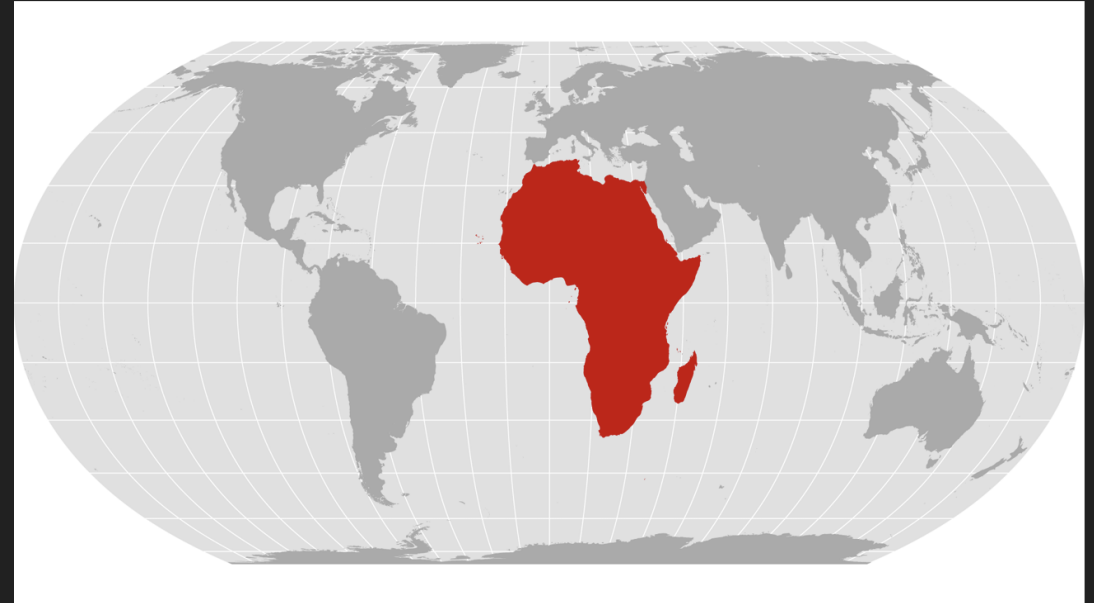
In the News

"Equal-area maps do not make Africa larger; they remove the exaggeration Mercator gives landmasses closer to the poles."

The U.N. backs a new world map showing Africa in its true relative size

SEPTEMBER 4, 2026 · 6:26 PM ET

By [Jewel Bright](#)



Alyson Hurt/NPR



UN News

@UN_News_Centre

On Friday, the United Nations General Assembly adopted an African proposal for a new world map, which shows countries' true size relative to one another.

In favor: 164

Against: 1

Abstentions: 6

Reminder

Group Formation (Today!)

Sep 9 – 12:00 pm

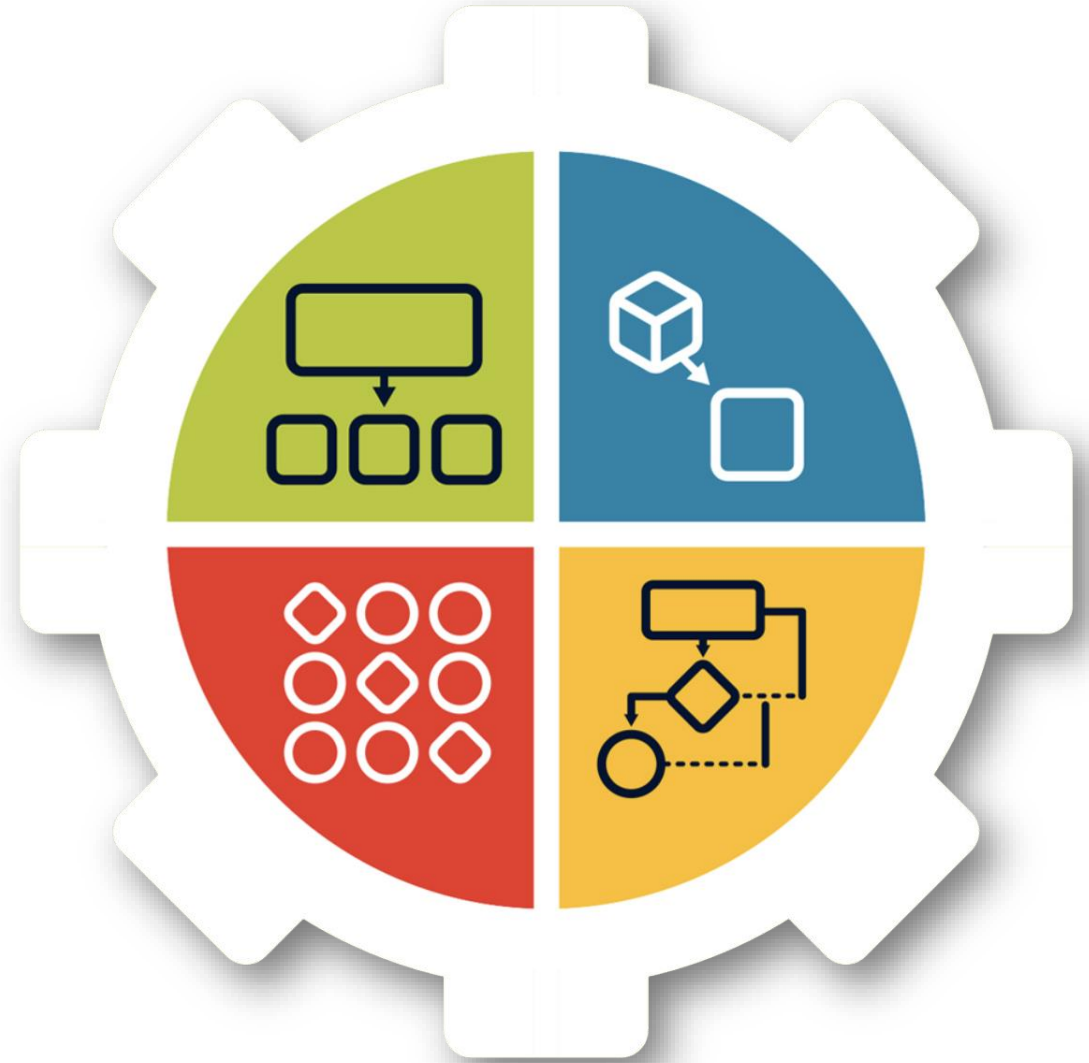
Assignment 1 released

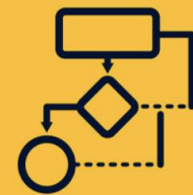
Due: Wed, Sep 16 · 11:59 PM

Programming

Computational Thinking Principles

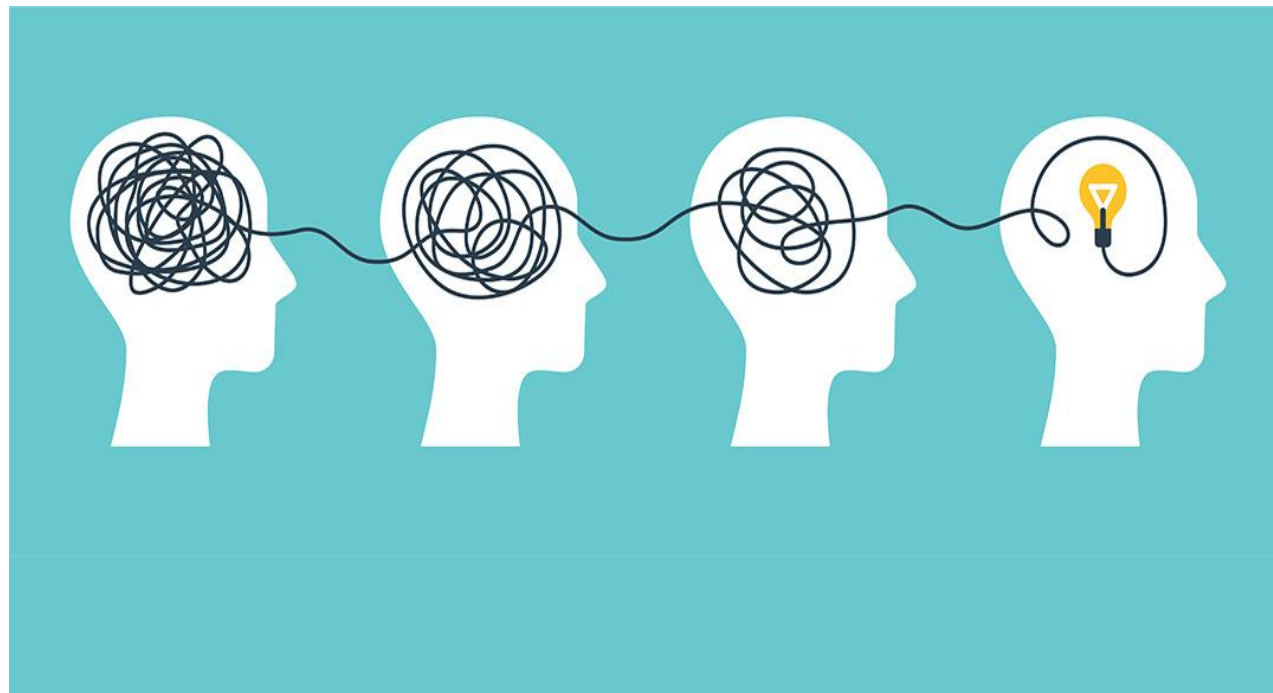
- Decomposition
- Abstraction
- Pattern Recognition
- Algorithm





Algorithm

- Step-by-step instructions for solving a problem which can be used for similar problems.
- Following the algorithm's steps correctly ensures arriving at the correct solution, regardless of the input.





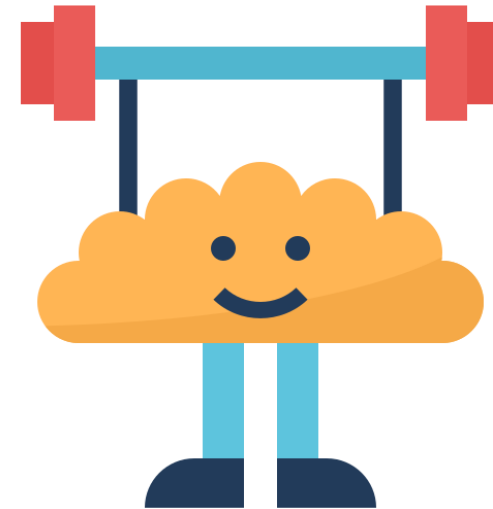
Code is an implementation of an algorithm



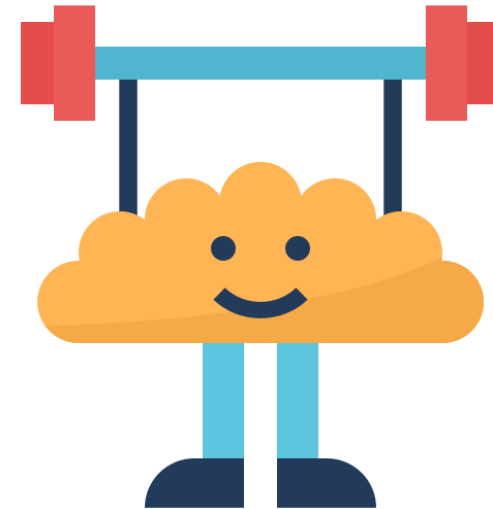
Turing Complete:

Anything computable in one language, is computable in any other programming languages.

Learning to code is
like
building muscles

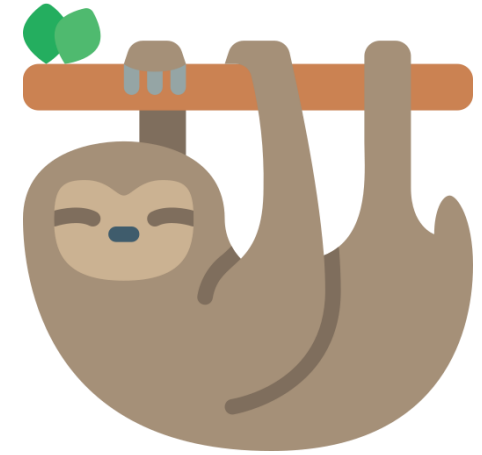


Get Comfortable with Being Uncomfortable

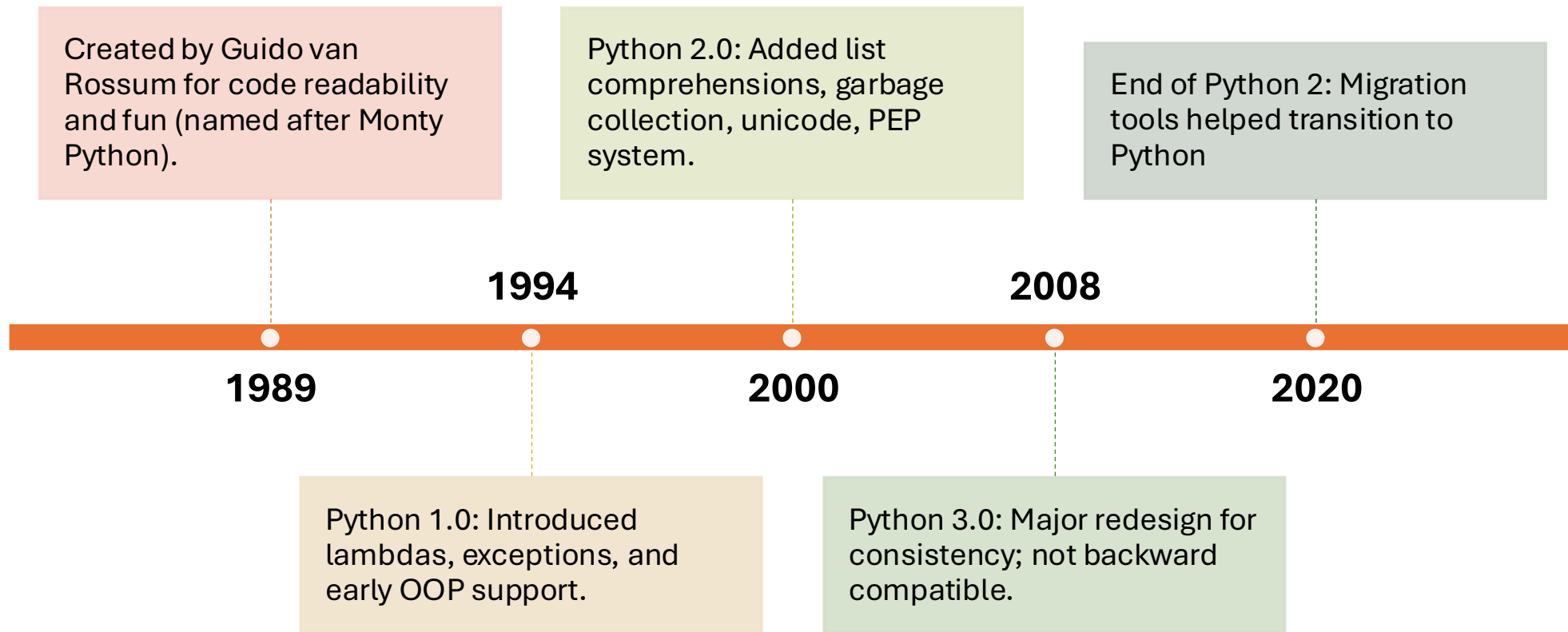


It's All About Laziness

- How can I make the computer do the work for me?
- What's the fastest way to get it done?
- How can I avoid repetitive tasks?
- How can I write code that's easy to understand, so I save time when revisiting it later and don't have to spend hours explaining it to others?
- How can I write a program that takes the least time to test and debug?



Python Basics



Built-in Functions

```
print("Hello, World!")
```

print() is a **built-in** function

Python **built-in** functions are pre-defined functions provided by the Python language that can be used to perform common tasks.

Built-in Functions

```
print("Hello, World!")
```

print() is a built-in function that prints any non-keyword **argument** into a file stream.

Our Brain can read through typos

"According to a research at Cambridge University, it doesn't matter in what order the letters in a word are, the only important thing is that the first and last letter be at the right place. The rest can be a total mess and you can still read it without problem. This is because the human mind does not read every letter by itself, but the word as a whole."

Computers take everything literally!

- They interpret code **exactly as written**, without understanding intent or context.
- **No Guesswork:** Unlike humans, computers won't infer meaning from mistakes, every symbol, space, and word must be precise.
- **Strict Rules:** Even small errors like a missing semicolon or misaligned indentation can break the entire program.
- **Think Like a Computer:** Writing proper syntax teaches you to communicate instructions clearly and logically.

Python syntax is case sensitive

Syntax: the symbols, words, and rules that make up valid Python code.

```
Print("Hello, World!")
```

Traceback (most recent call last):

File "<input>", line 1, in

NameError: name 'Print' is not defined. Did you mean: 'print'?

Syntax is Like LEGO Pieces





Bugs



What is a Bug?

- A bug is an error or mistake in a program, and it can appear in many forms.
- You'll encounter plenty of bugs—just like I have and still do!
- Bugs are problems for you to solve that help you grow as a programmer.
- With time, practice, and experience, you'll become more confident in handling them.

Embrace the Errors!

- Errors are part of the journey to becoming a better programmer.
- **Expect them**, they're how you grow.
- Debugging builds problem-solving skills and confidence.
- Become friends with Error messages. **Read them**, know different errors, and document your problem solving strategies.
- We don't want just any code, we want **readable, testable** code.

Take Five



Python Diagnostics Survey

The grade shown at the end is for diagnostic purposes only and does not affect your course grade.

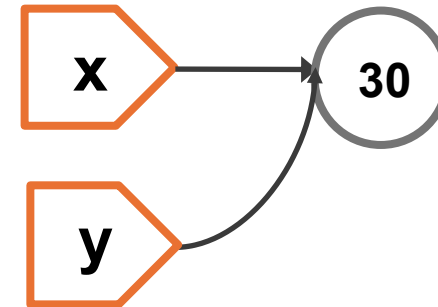
Enter your Berkeley email address.

If you have never worked with Python before, the survey will record your background and will not ask you to answer Python questions.



Variables

- A variable is a **name** that **refers to** a value.
- In Python, it is often helpful to think of a variable as a **label attached to an object**.
- Multiple variables can refer to the **same object**.



Assignment / Binding

- Variables are created by using an **equal sign** `=`, also known as **assignment operator**.

age = 25

- The act of associating a name on the left to a value on the right is called assignment or binding.
- The assignment operator does not copy anything!

Naming Variables

1. Cannot start with a number:

Invalid: `1name = "Sara"`

Valid: `name1 = "Sara"`

2. Avoid starting with uppercase letters (convention)

Uppercase names are typically reserved for constants or class definition.

Preferred: `name = "Tina"`

Valid but unconventional: `Name = "Tina"`

Naming Variables

3. Allowed characters: Letters (a-z, A-Z) , Numbers (0-9) , Underscores (_)

Valid: `first_name = "Tina"`

Valid: `firstName = "Tina"`

Invalid: `&first_name = "Tina"`

Invalid: `first.name = "Tina"`

4. Cannot use **reserved names or keywords**. Reserved words have predefined meanings in Python and cannot be used as variable names such as **if, else, for, while, True, False, class**

Valid: `condition = 8.5`

Invalid: **`if`** `= 8.5`

Reserved Words

The following reserved words **cannot be used** as **variable** names

. False	assert	else	import	pass
. None	break	except	in	raise
. True	class	finally	is	return
. and	continue	for	lambda	try
. as	def	from	nonlocal	while
. await	del	global	not	with
. async	elif	if	or	yield

Choosing Proper Names

```
. x1q3z9ocd = 35.0  
. x1q3z9afd = 12.50  
. x1q3p9afd = x1q3z9ocd * x1q3z9afd  
. print(x1q3p9afd)
```

```
a = 35.0  
b = 12.50  
c = a * b  
print(c)
```

```
hours = 35.0  
rate = 12.50  
pay = hours * rate  
print(pay)
```

What is this bit of code
doing?

Is this easy to read?

Data Types in Python

	Data Type	Definition	Example
Integer	int	Integer numbers, whole numbers without a fractional part.	42, -5, 0
Float	float	Floating-point numbers, numbers with decimal points.	3.14, -0.01, 0.0
String	str	Strings, sequences of characters enclosed in single or double quotes.	'hello' , "Python"
Boolean	bool	Boolean values, representing True or False.	True, False

Numeric Type Operations

Operation	Result
$x + y$	sum of x and y
$x - y$	difference of x and y
$x * y$	product of x and y
x / y	quotient of x and y
$x // y$	floored quotient of x and y
$x \% y$	remainder of x / y
$-x$	x negated
abs(x)	absolute value or magnitude of x
int(x)	x converted to integer
float(x)	x converted to floating point
pow(x, y)	x to the power y
$x ** y$	x to the power y

Comparison Operators

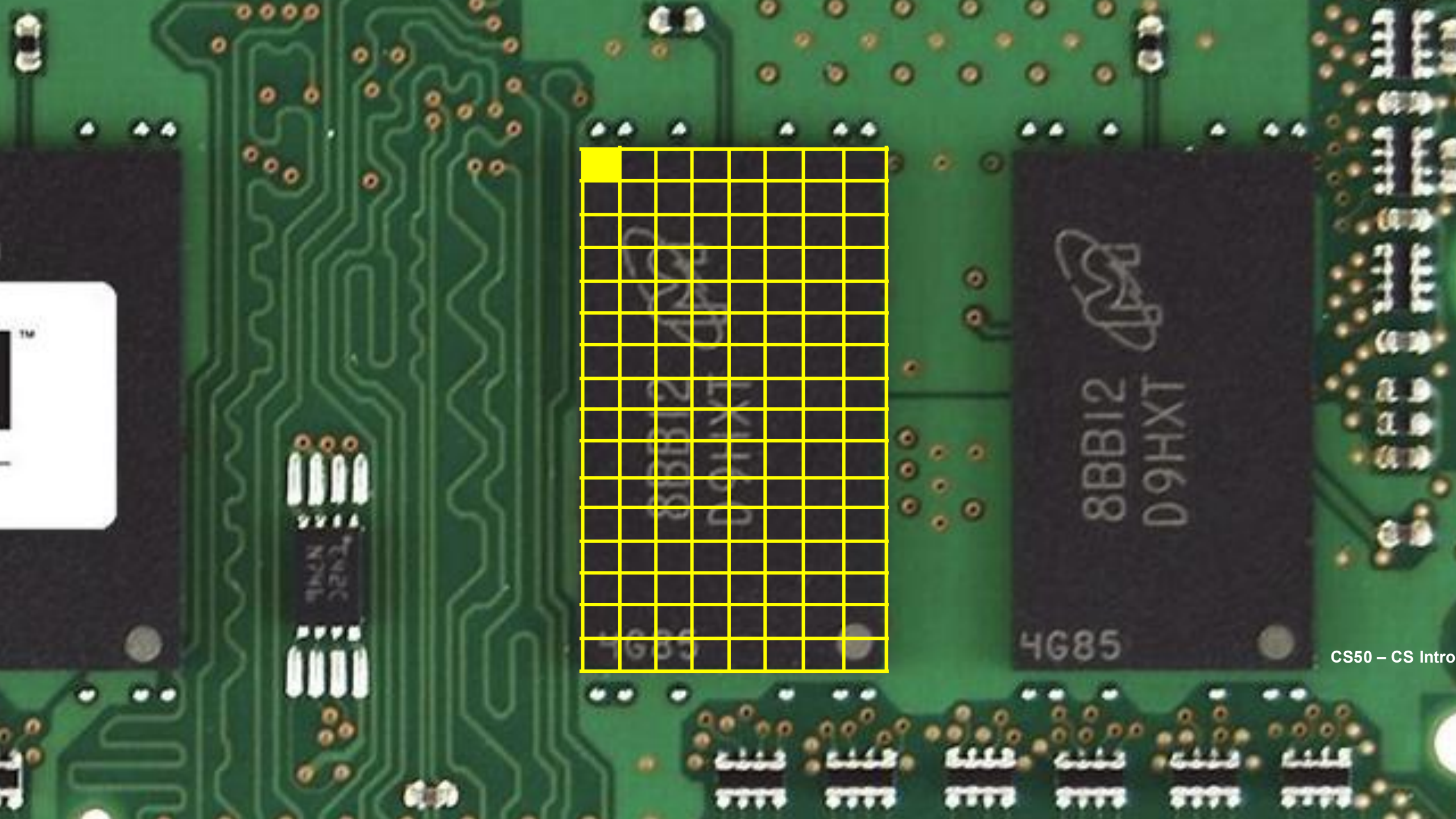
Boolean Return:

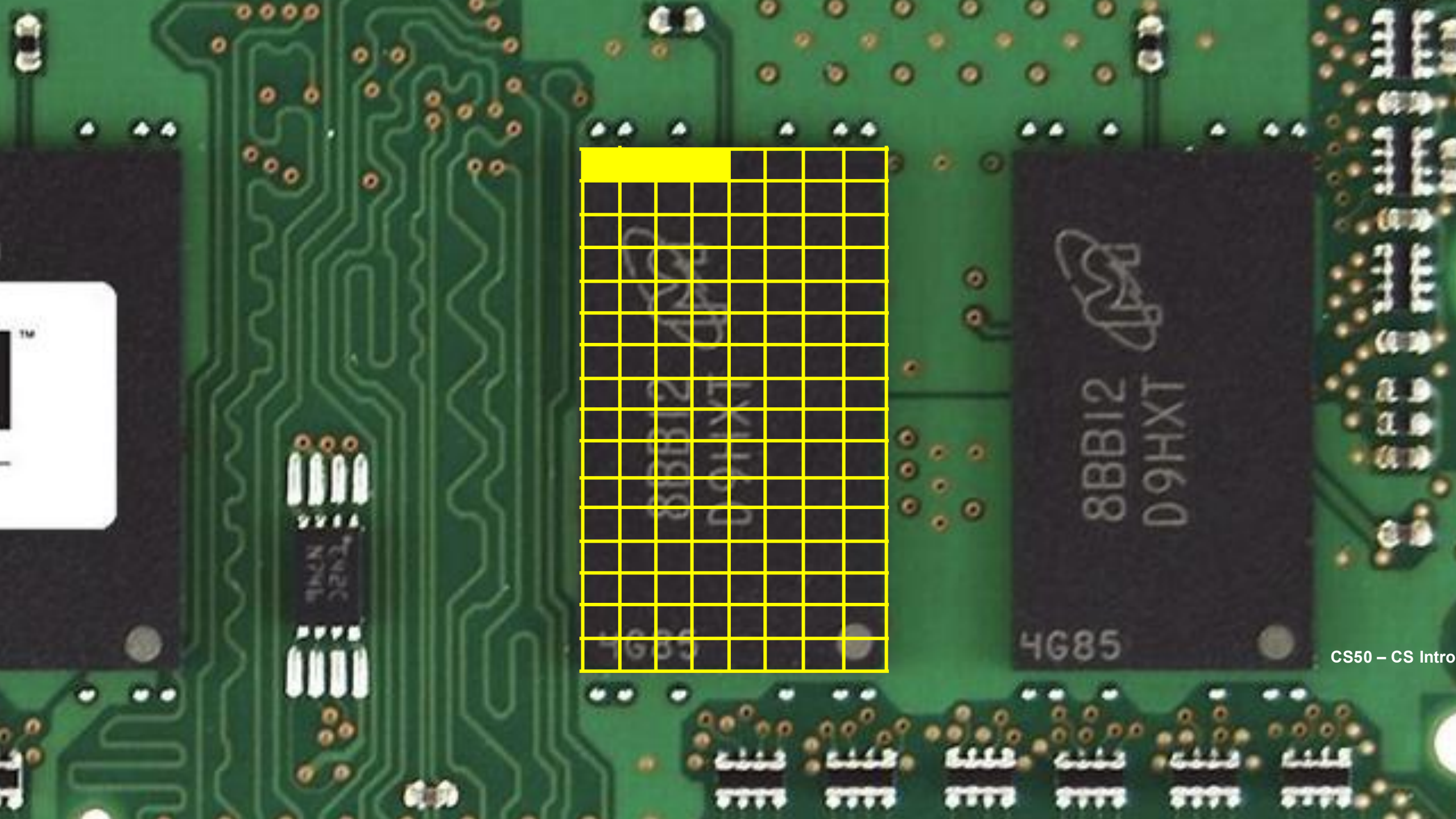
True

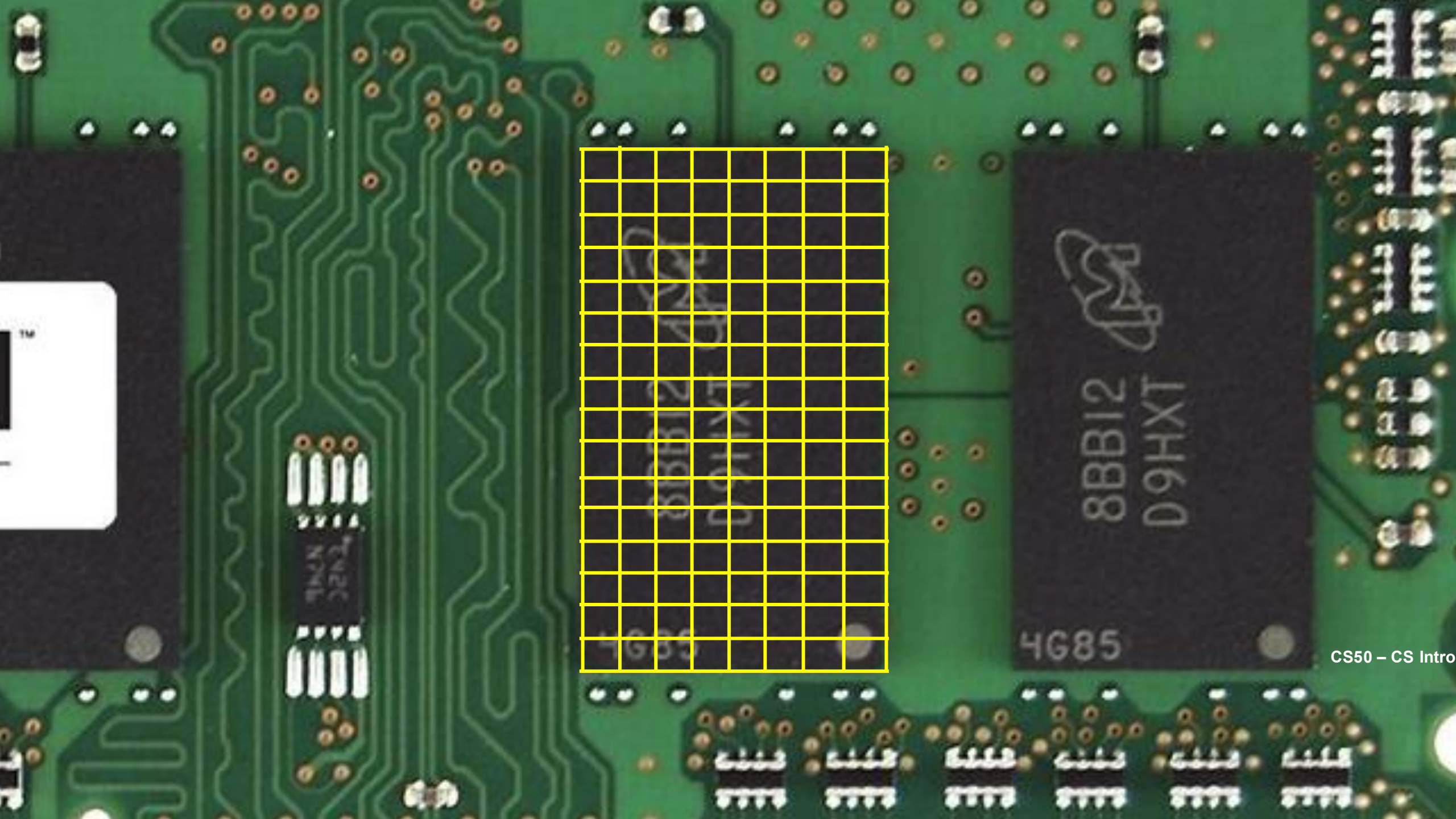
False

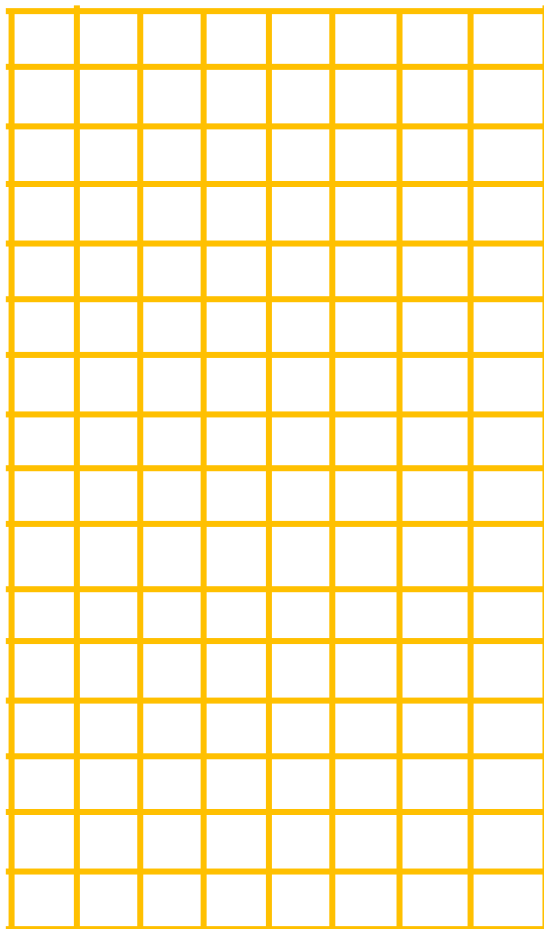
Operation	Meaning
<	strictly less than
<=	less than or equal
>	strictly greater than
>=	greater than or equal
==	equal
!=	not equal
is	object identity
is not	negated object identity





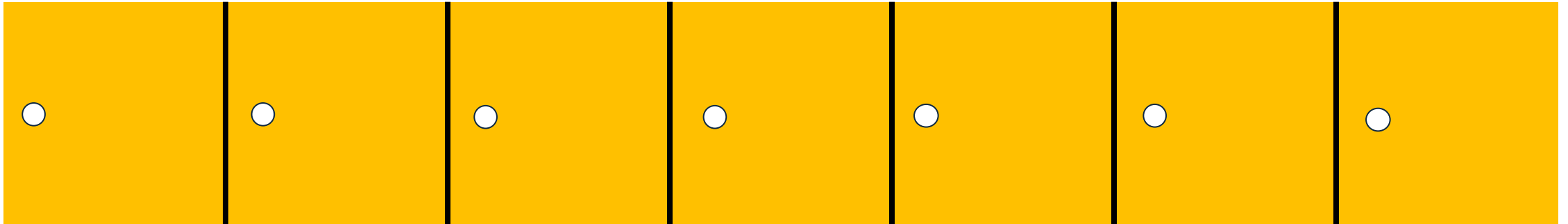






--	--	--	--	--	--	--

1	15	10	2	50	15	67
---	----	----	---	----	----	----





Data Structures in Python

Data Structures in Python

Type	Example	Mutable	Ordered	Uniqueness
list	[1, 2, 3]	Yes	Yes	Duplicates allowed
tuple	(1, 2, 3)	No	Yes	Duplicates allowed
set	{1, 2, 3}	Yes	No	only unique
dict	{"a": 1}	Yes	Yes	Keys unique

Sequence

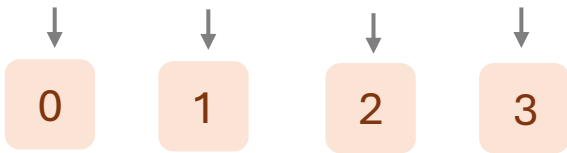
- Sequences are **ordered** collections. They maintain the order of the things in them.
- Sequences are a special type of **iterable** that can be indexed using square brackets ([...]) to get items by their position.
- We can use the **len()** function to see how many things are inside them.
- Sequences can be indexed from 0 up until index `len(<my_sequence>) - 1`
- In Python **list**, **tuple**, and **string** are sequences

```
>>> greeting = "Hi y'all!"
>>> fruits = ['apple', 'lemon', 'pear', 'watermelon']
>>> coordinates = (1, 8, 2)
```

list

Lists in Python are ordered collections of objects.

```
>>> ped_counts = [16, 23, 16, 11] repetition is allowed
```

index ←  Index start from 0 in python

```
>>> ped_counts[3]  
11
```

```
>>> ped_counts[4]  
?
```

`IndexError: list index out of range`

list

Lists can contain all types of objects; they don't have to have all strings, integers, etc. (though in practice, we usually use lists with a single data type)

```
>>> things = [5, True, "hello"]
```

You can even put lists of things in other lists!

```
>>> more_things = [things, ped_counts]
```

```
>>> more_things
```

```
[[5, True, 'hello'], [16, 23, 16, 11] ]
```

```
>>> len(more_things)
```

```
2
```

list - Slicing

We can also get a portion of things from sequences, we call it slicing.

Slicing follows the format:

`sequence[start:stop:step]`

- start is **inclusive** (included in the result),
- stop is **exclusive** (not included in the result),
- step (optional) determines the increment between elements.

```
>>> ped_counts = [16, 23, 10, 11, 16, 42, 9, 0, 14, 37]
```

```
>>> ped_counts[1:7:2]
```

```
[23, 11, 42]
```

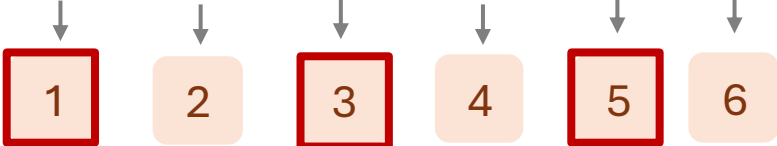
list

```
>>> ped_counts = [16, 23, 10, 11, 16, 42, 9, 0, 14, 37]
```



index	0	1	2	3	4	5	6	7	8	9
value	16	23	10	11	16	42	9	0	14	37

```
>>> ped_counts[1:7:2] ?
```



index	1	2	3	4	5	6
value	23	10	11	16	42	9

```
[23, 11, 42]
```

list

```
>>> things = [5, True, "hello"]  
>>> ped_counts = [16, 23, 16, 11]
```

What if I want to add all the elements of `ped_counts` to the `things` list instead of having two lists in one?



What is this?
A function??

```
>>> things.extend(ped_counts)  
>>> things  
[5, True, 'hello', 16, 23, 16, 11]
```

list - Methods

Each object comes with its own built-in menu

- A **method** is a function that *belongs to* a specific data type. An action that type knows how to perform.
- A **variable** is a noun (a thing: an object)
- A **method** is a verb (an action the object can do).
- The **dot** connects an object to its methods.

```
>>> things = [5, True, "hello"]
```

```
>>> things.reverse()
```

```
>>> things ?
```

```
['hello', True, 5]
```

list

```
>>> things = [5, True, "hello"]
```

Add items to a list (to the end of the list): `.append()` method

```
>>> things.append("end")
```

```
>>> things
```

```
[5, True, "hello", "end"]
```

Get the last item from a list and remove it from that list at the same time: `.pop()` method

```
>>> things = [5, True, "hello"]
```

```
>>> things.pop()
```

```
"hello"
```

```
>>> things
```

```
[5, True]
```

list

We can check whether a list contains a particular item by using the `in` operator. The `in` operator returns `False` if the list doesn't contain that item, and it returns `True` if it does:

```
>>> things = [5, True, "hello"]
```

```
>>> "hello" in things
```

```
True
```

```
>>> 10 in things
```

```
False
```

```
>>> "5" in things
```

```
?
```

tuple

Tuples are like lists, but **immutable**, meaning they can't be mutated (changed in place).

Like a list, a tuple can be indexed:

```
>>> coordinates = (3, 4, 5)
```

```
>>> coordinates[0]
```

```
3
```

A tuple can be sliced:

```
>>> coordinates[1:]
```

```
(4, 5)
```

tuple

```
>>> coordinates = (3, 4, 5)
```

```
>>> type(coordinates)
```

```
<class 'tuple'>
```

```
>>> coordinates = 3, 4, 5
```

```
>>> type(coordinates)
```

```
?
```

```
<class 'tuple'>
```

tuple

You can get the **length** of a tuple:

```
>>> len(coordinates)
3
```

Unlike a list though, ***you cannot replace, add, or remove elements after the tuple is created:***

```
>>> coordinates[0] = 1
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

TypeError: 'tuple' object does not support item assignment

str



- A string is a **sequence** of characters
- Strings are **immutable**
- A string literal uses quotes : 'Hello' or "Hello"
- With strings, **+** joins (*concatenates*) them together.
- When a string contains numbers,
it is still a string

We can convert numbers in a string into numeric types using `int()` or `float()`:
`int("5"), float("0.2")`

str - indexing

```
>>> city = "New York"
```

```
>>> city[0]
```

```
'N'
```

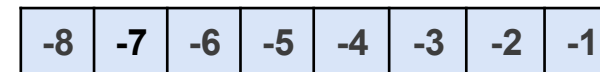
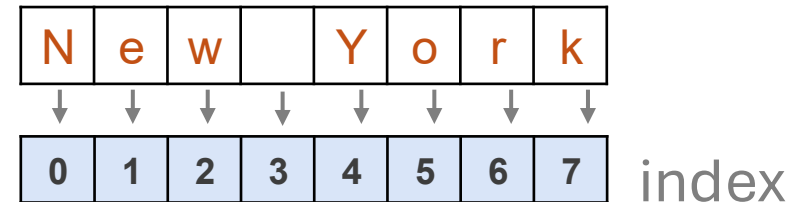
```
>>> city[3]
```

```
?
```

```
>>> city[-1]
```

```
?
```

Indices start from 0 in python



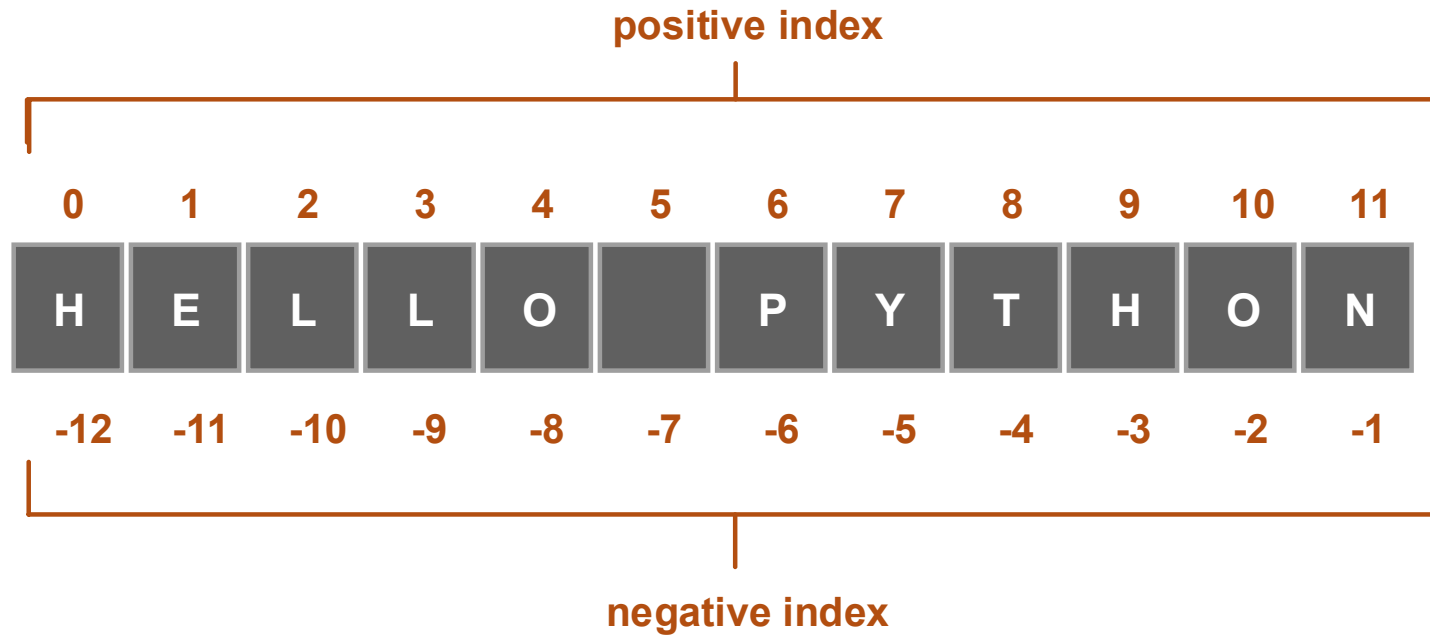
how many characters are in the city variable?

```
>>> len(city)
```

```
8
```

str - indexing

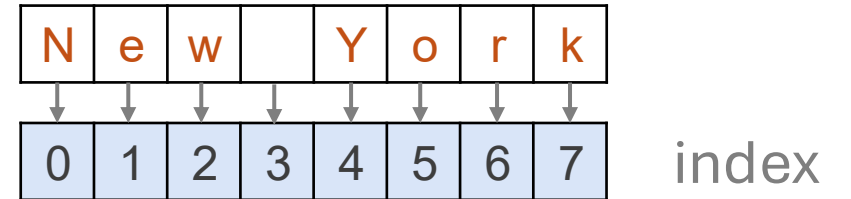
String indexing in Python



Indices count from the left with 0, 1, 2, ... and from the right with -1, -2, -3, ...

str - Slicing

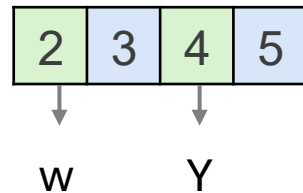
Slicing follows the format:
sequence[start:stop:step]



```
>>> city = "New York"
```

```
>>> city[2:6:2]
```

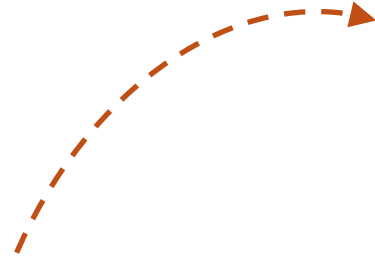
```
'wY'
```



Python Standard Library

- The **Python Standard Library** is a collection of modules that are distributed with Python and provide access to a wide range of functionalities without needing to install **external packages**.
- External packages are like board games' **expansion packs**.





Hello, World!

```
print("Hello, World!")
```

print() is a **function**

```
view_state = {  
    "longitude": -73.98416810282863,  
    "latitude": 40.72651721370669,  
    "zoom": 12.726630492730596,  
    "pitch": 59.80465353190481,  
    "bearing": 13.243243243243244,  
}  
m = Map(layer, view_state=view_state)  
m
```

